Fiverr AI Seller Growth Platform

Technical Stack & Architecture Reference

Version: 1.0 | **Date:** October 2026 | **Backend:** Supabase

1. Product scope

The platform retains all three original workflows:

1. **Existing Gig Optimizer:** User supplies Fiverr username, gig URL, gig content, and optionally seller analytics; generate an SEO audit, revised content, and recommendations.

2. **AI Gig Creator:** Collect service information and generate title options, description, tags, category suggestions, FAQs, buyer requirements, Basic/Standard/Premium packages, and image briefs.

3. **Skills-to-Service Opportunity Finder:** Collect skills and experience; suggest specialized Fiverr services with explainable scores, evidence and confidence labels. Never present a score as a probability of receiving an order.

Additional modules: Gig Problem Doctor; thumbnail/mobile preview tester; competitor gap analysis; buyer reply coach; gig version and experiment tracking; portfolio/demo planner; pricing and profitability calculator; personalized growth roadmap; performance analytics.

2. Recommended stack

Layer | Recommended technology | Responsibility

Web framework | Next.js 16 App Router | Pages, server rendering, route handlers

Language | TypeScript | End-to-end type safety

Styling | Tailwind CSS | Responsive design

UI | shadcn/ui + Lucide React | Accessible components and icons

Animation | Motion / Framer Motion | Optional interactions

Database | Supabase PostgreSQL | Relational data and reporting

Auth | Supabase Auth + `@supabase/ssr` | Email/OAuth sessions

Authorization | PostgreSQL Row Level Security | Per-user access controls

Files | Supabase Storage | Private screenshots, thumbnails, PDFs

Server endpoints | Next.js Route Handlers / Server Actions | Validation, orchestration, secure API access

Edge functions | Supabase Edge Functions | Lightweight webhooks and events

AI | OpenAI API; optional Anthropic | Structured audits, suggestions, image understanding

Validation | Zod + JSON Schema | User input and model-output validation

Forms | React Hook Form + Zod | Form handling

Fetch/cache | TanStack Query | Interactive client state

Local state | Zustand (only if needed) | UI state

Charts | Recharts | Seller metrics and experiments

Long jobs | Inngest or Trigger.dev | Async analysis and retries

Billing | Regionally supported provider, Stripe where eligible | Plans, credits, webhooks

Monitoring | Sentry | Error reporting

Hosting | Vercel + Supabase | Web and managed backend

Architecture choice: Supabase is the backend platform; Next.js provides trusted server-side application logic. A separate Express server is not needed for MVP. Do not expose service-role keys or AI secrets to browsers.

3. High-level request flow

```
Browser (Next.js UI)
  |-- Supabase Auth (session)
  |-- Next.js server routes (authorization, Zod validation, quotas)
       |-- Supabase PostgreSQL (RLS)
       |-- Supabase Storage (private buckets, signed URLs)
       |-- AI provider (server-side only)
       |-- Background job runner (large/slow analyses)
       |-- Billing provider (signed webhooks)
```

4. Core Supabase tables

- `profiles`: `id` (auth.users FK), display_name, onboarding_stage, timestamps.

- `user_skills`: user_id, skill, level, years_experience.

- `seller_profiles`: user_id, marketplace, username, profile_url.

- `gigs`: id, user_id, seller_profile_id, source_url, service_name, status, current_version_id.

- `gig_versions`: id, gig_id, title, description, category, tags JSONB, packages JSONB, faqs JSONB, created_at.

- `gig_audits`: id, gig_id, version_id, quality_score, findings JSONB, recommendations JSONB, model_version, created_at.

- `gig_images`: id, gig_id, storage_path, audit JSONB.

- `gig_metrics`: id, gig_id, period_start, period_end, impressions, clicks, orders, revenue, source.

- `service_opportunities`: id, user_id, service_title, skill_match, opportunity_score, confidence, rationale JSONB.

- `research_evidence`: id, opportunity_id, source_type, source_url, observed_at, metrics JSONB, limitations.

- `competitor_analyses`: id, user_id, gig_id, supplied_competitor_data JSONB, findings JSONB.

- `buyer_messages`: id, user_id, original_text, generated_draft, created_at.

- `portfolio_projects`: id, user_id, brief JSONB, artifacts JSONB.

- `growth_roadmaps`: id, user_id, gig_id, summary, created_at.

- `roadmap_tasks`: id, roadmap_id, priority, action, status.

- `ai_jobs`: id, user_id, job_type, status, input_ref, output_ref, error, timestamps.

- `subscriptions`: id, user_id, provider, external_subscription_id, plan, status.

- `usage_events`: id, user_id, feature, units, job_id, created_at.

Add foreign keys, appropriate unique constraints, indexes on user_id and gig_id, and timestamps. Keep evidence distinct from AI conclusions. Avoid unnecessary duplication of packages until relational queries justify separate tables.

5. Security rules

1. Enable RLS on every user-owned table. A user should only access rows with `user_id = auth.uid()`; child tables require ownership checks through parent rows.

2. Use Supabase SSR clients and server-side session validation. Treat browser-provided user IDs as untrusted.

3. Keep OpenAI, Anthropic, service-role and billing keys server-side. Prefer least-privilege credentials.

4. Store user uploads in private buckets with path/ownership policies; issue short-lived signed URLs.

5. Validate MIME type, size, dimensions, rate limits, quotas and structured AI output.

6. Verify payment webhook signatures and handle retries idempotently.

7. Minimize personal data in buyer-message inputs; provide deletion and retention controls.

8. Use row ownership and credit checks before expensive AI calls; log job status and cost.

6. AI design by module

Module | Inputs | Processing | Output

Gig optimizer | User-provided gig text, screenshots, URL identifier | Rule checks + LLM analysis | Audit, revised gig draft

Gig creator | Service, deliverables, audience, expertise | Structured generation + rule validation | Copy-ready gig sections

Opportunity finder | Skills, goals, permitted research | Candidate generation + explainable weighted scoring | Ranked ideas with evidence/confidence

Problem doctor | Impressions, clicks, orders, dates | Deterministic CTR and conversion calculations + explanations | Likely bottlenecks and tests

Thumbnail tester | User-uploaded images | Vision review + mobile-size previews | Readability, cropping and clarity feedback

Competitor gaps | User-supplied/permitted listing details | Compare offers and positioning | Differentiated service angles

Buyer coach | Pasted buyer message | Draft with scope clarification | Editable professional reply

Experiment tracker | Gig versions and metric periods | Before/after comparisons with caveats | Trends, change history

Portfolio planner | Skills and chosen niche | Demo brief and proof checklist | Honest sample project plan

Profitability | Price, time, expenses, platform fees | Deterministic arithmetic | Net and hourly earnings estimate

Growth roadmap | Audit and seller goals | Prioritize actions | Ordered, trackable tasks

Scoring guidance: An opportunity score may weight demand evidence 30%, competitive accessibility 25%, skill fit 20%, pricing potential 15%, differentiation 10%. These weights are a starting hypothesis, not validated predictive probabilities. Always display source quality and uncertainty.

7. Data access and Fiverr compliance

- Do not rely on unauthorized Fiverr scraping, CAPTCHA bypass, proxy rotation, or automatic account manipulation.

- Accept URLs as identifiers and ask users to provide gig text, screenshots, and exported analytics. Investigate official permitted integration options before automated retrieval.

- No claim that SEO improvements guarantee search placement or orders. Fiverr rankings are affected by multiple factors.

- Use user-confirmed output; do not auto-post gigs or messages.

- Verify current Fiverr field limits, marketplace rules and branding restrictions before implementing validators.

8. Suggested repository structure

```
src/
  app/
    (auth)/login/  (auth)/register/
    (dashboard)/dashboard/
    (dashboard)/discover/
    (dashboard)/create-gig/
    (dashboard)/optimize-gig/
    (dashboard)/gig-doctor/
    (dashboard)/thumbnails/
    (dashboard)/competitors/
    (dashboard)/experiments/
    (dashboard)/buyer-coach/
    (dashboard)/portfolio/
    (dashboard)/growth-roadmap/
    (dashboard)/billing/
    api/ai/  api/gigs/  api/opportunities/  api/metrics/  api/webhooks/
  components/ui/  components/dashboard/  components/gigs/
  lib/supabase/client.ts
```

```
    lib/supabase/server.ts
    lib/supabase/admin.ts  # server-only
    lib/ai/  lib/scoring/  lib/validations/  lib/billing/
    hooks/  types/
  supabase/migrations/
  supabase/functions/
  inngest/  # if selected
  proxy.ts  # Next.js 16 convention, if needed
```

9. Core API endpoints (proposed)

- `POST /api/gigs`: create gig from user input.
- `POST /api/ai/gig-audit`: enqueue audit with authenticated ownership check.
- `POST /api/ai/gig-generate`: create structured draft.
- `POST /api/opportunities/discover`: rank niche candidates with confidence.
- `POST /api/ai/thumbnail-audit`: inspect user-owned upload.
- `POST /api/metrics`: save seller-entered metrics.
- `GET /api/gigs/:id/versions`: version history.
- `POST /api/ai/buyer-reply`: draft response, never send automatically.
- `POST /api/webhooks/billing`: verified, idempotent payment events.

Long-running endpoints should return a job ID and use background processing with retry limits and progress states.

10. MVP milestones

Phase 0 — Validate (week 1): interview 15–20 sellers, test report value and willingness to pay, confirm compliant research inputs.

Phase 1 — Foundation (week 2): Next.js, Supabase Auth, PostgreSQL schema, RLS, storage, onboarding, dashboard.

Phase 2 — Core MVP (weeks 3–4): original three features; manual gig import, structured outputs, copy-ready drafts, evidence labels.

Phase 3 — Differentiation (weeks 5–6): Problem Doctor, thumbnail preview, personalized roadmap, version tracking.

Phase 4 — Commercial beta (weeks 7–8): quotas, billing where supported, monitoring, feedback, secure exports.

Later: competitor gap analysis, buyer coach, portfolio builder, profitability tools, advanced experiment reporting.

Timelines are illustrative for a small scoped implementation, not delivery guarantees.

11. Testing and acceptance criteria

- Authenticated user cannot read/write another user's gig, report, image or metrics (RLS tests).
- Model outputs conform to Zod schemas and fail gracefully on invalid data.
- Duplicate AI job retries and payment webhooks do not double-charge credits.
- Uploaded files remain private; signed links expire.
- Analytics calculations handle zero clicks, missing dates, and small samples.
- Service recommendations show evidence, freshness and confidence rather than fabricated order probabilities.
- Responsive gig-image preview works on common mobile widths.

- All copy is user-editable before publication.

12. Cost and scaling notes

Start with Supabase and Vercel entry tiers if eligible. Meter AI tokens, image analysis and generated media separately. Cache deterministic audits by content hash when appropriate. Use queues for expensive operations, job concurrency limits, user quotas and observability. Assess regional payment eligibility before choosing a billing provider.

13. Suggested development priorities

Must-have: Auth/RLS, onboarding, original three features, manual input, structured validation, saved gigs, credits and logging.

High-value next: Problem Doctor, image preview, roadmap, experiment tracker.

Later: competitor intelligence from permitted sources, buyer coach, portfolio and profitability modules, marketplace expansion.

14. Primary documentation

- Supabase docs: https://supabase.com/docs
- Supabase SSR: https://supabase.com/docs/guides/auth/server-side/nextjs
- Supabase RLS: https://supabase.com/docs/guides/database/postgres/row-level-security
- Next.js docs: https://nextjs.org/docs
- OpenAI API docs: https://platform.openai.com/docs
- Fiverr Terms: https://www.fiverr.com/terms_of_service

Decision: Use Next.js 16 + TypeScript + Supabase Auth/PostgreSQL/RLS/Storage + OpenAI + Zod as the MVP core. Add background jobs, billing, and analytics tools when feature needs justify them.